

R5 — O pino que não responde, e o conversor

A fronteira analógico/digital, quantização e o que a média não conserta

Microcontroladores — DENE/UFMT

OBJETIVOS

- Explicar por que o botão responde sem configuração, descobrindo o estado que o bootloader deixa no conversor.
- Provocar e diagnosticar o sintoma do botão que não responde, e corrigi-lo pela causa.
- Medir o degrau do conversor na própria bancada e comparar com o valor derivado.
- Converter o código de dez bits em décimos de grau, sem ponto flutuante.
- Distinguir ruído aleatório de erro de quantização, e decidir o que a média resolve.

CH2-1 (LCD)	ON — o display do R4 continua	NESTE KIT
Chaves SWITCHS (PORTB)	OFF	
Botão	SW12, marcado INT0, em RB0 — ativo em nível baixo; inferior direito, ao lado do RESET	
LEDs de PORTD	ativos em nível baixo: LATD0 = 0 acende	
CH5-3 e CH5-4	OFF	
CH1-7 (TEMP)	ON — LM35 em RA0/AN0	
CH1-1, CH1-5 e CH1-6	OFF — também chegam ao RA0	

A verificar antes da sessão: a tensão real da alimentação, medida com o multímetro. O fundo de escala do conversor é a própria alimentação, e todo número da Parte 2 depende dela. Para a tabela completa de PCFG e o mapa de registradores, consulte a **folha de referência do ADC** na página do curso.

Pontuação. As sete tarefas somam 8,0, e a folha de previsões entregue preenchida vale 2,0. A nota de cada tarefa está na margem, ao lado dela. A seção *Se sobrar tempo*, no fim, não vale nota: é para quem terminar antes.

1 Parte 1 — o botão que responde, e por quê

Onde fica o botão, e em que pino ele chega. Na seção PUSH BUTTONS, na metade inferior da placa, seis botões em três linhas de dois. O INT0 (SW12) é o **inferior direito** — e o vizinho imediato dele, à esquerda, é o RESET (SW9):

BANCADA

	Coluna esquerda	Coluna direita
Linha 1	CH0 (SW11)	INT2 (SW14)
Linha 2	TMR1 (SW10)	INT1 (SW13)
Linha 3	RESET (SW9)	INT0 (SW12)

Eletricamente, o SW12 chega ao **RB0** do microcontrolador — o mesmo pino que o datasheet também chama de **AN12**. Guardem esses dois nomes para o mesmo pino: é disso que trata a Parte 1 inteira.

E cuidado com o vizinho. Apertar o RESET sem querer reinicia o programa, e o sintoma disso se parece com um defeito de código — o contador zera, o display volta ao início, o LED apaga. Antes de culpar o programa, confira qual dos dois botões o seu dedo alcançou.

Tarefa 1. Complete o programa abaixo, para que o LED de RD0 acenda enquanto o botão INT0 estiver pressionado. **Não escreva nada em ADCON1** — é justamente o que esta parte vai investigar.

TAREFA · 0,5 pt

```
#include <xc.h>

void main(void)
{
    /* Nenhuma linha de ADCON1 aqui. */
    LATDbits.LATD0 = 1; /* LED apagado: PORTD é ativo em nível baixo */
    TRISDbits.TRISD0 = 0; /* RD0 como saída */
    TRISBbits.TRISB0 = 1; /* RB0 como entrada: é lá que chega o SW12 */

    while (1) {
        /* Uma linha só, aqui: o botão pressionado leva RB0 a 0,
           e o LED aceso pede LATD0 = 0. */
    }
}
```

Grave, pressione o botão, e registre:

O que a previsão P1 dizia

O que o LED faz com o botão solto

O que o LED faz com o botão pressionado

O datasheet diz que RB0 é também o canal analógico AN12, e que um canal analógico habilitado não lê nível lógico. Mesmo assim, o botão respondeu. Alguém configurou esse pino antes de você.

Tarefa 2. Descubra o que o conversor herdou. Grave um programa cuja **primeira** linha copie ADCON1 para uma variável e a mostre nos LEDs:

TAREFA · 1,0 pt

```
unsigned char herdado = ADCON1; /* 1a linha */
LATD = (unsigned char)~herdado; /* aceso = 1 */
TRISD = 0x00;
while (1);
```

bit	7	6	5	4	3	2	1	0
nome	—	—	VCFG1	VCFG0	PCFG3	PCFG2	PCFG1	PCFG0
lido								

Valor de ADCON1 em hexadecimal: _____

Com a tabela de PCFG (na folha de referência do ADC): quais canais estão analógicos e quais digitais? Em qual grupo está RB0 (AN12)?

O datasheet dá **dois** valores de PCFG após o reset, e quem escolhe entre eles é o bit de configuração PBADEN. Qual dos dois vocês leram? Então quanto vale PBADEN neste kit?

O que os bits VCFG dizem sobre a referência do conversor?

Vocês não escreveram PBADEN, e não poderiam: os bits de configuração pertencem ao bootloader, e o `#pragma config` da aplicação é ignorado. Quem gravou o

CONCEITO

bootloader escolheu PBDEN = 0, e o próprio reset carregou 0111 em PCFG. Não há código escondido escrevendo ADCON1: há um bit escolhido por outra pessoa, e o hardware obedecendo a ele.

É o princípio do encontro 0 pela terceira vez — *o código que veio antes do seu* —, agora na forma mais traiçoeira: um programa que **funciona** por causa de uma decisão que você não tomou, não vê e não controla.

Tarefa 3. Agora provoque o sintoma. Acrescente, antes do laço da Tarefa 1:

TAREFA · 1,0 pt

```
ADCON1 = 0x00; /* tudo analógico, até AN12 */
```

Antes de gravar, preveja o que o LED fará nas duas posições do botão. Depois grave e registre:

Previsão

Botão solto

Botão pressionado

Antes de corrigir, meça: com o multímetro, verifique a tensão no pino do botão nas duas posições.

O pino está mudando de tensão? _____

Então o defeito está no circuito ou no programa? _____

O pino muda de tensão e o programa lê sempre zero. O botão funciona, o fio funciona, e a leitura não.

CONCEITO

A causa está entre os dois: quando o canal analógico de um pino está habilitado, o **buffer de entrada digital é desligado**, e a leitura de PORTB devolve 0 independentemente do que houver no fio. Como o LED acende com 0, ele fica aceso o tempo todo: o sintoma aparece como “o LED não apaga”, e não como “o botão não funciona”.

Tarefa 4. Troque 0x00 por 0x0F, grave, e confirme que o botão voltou.

TAREFA · 1,0 pt

Neste kit, o programa da Tarefa 1 funcionava sem essa linha. Por que, então, o padrão do curso passa a ser escrever ADCON1 = 0x0F na primeira linha?

Com o valor herdado 0x07, cite dois pinos do kit que **ainda** sofreriam o sintoma se fossem usados como entrada digital.

Por que este defeito é difícil de encontrar sozinho? Cite uma hipótese errada que um aluno testaria antes de chegar na causa.

E o padrão do curso a partir de agora: **tudo digital na primeira linha**, e o analógico se declara explicitamente depois, habilitando só o canal que se vai usar. Não é superstição: é não depender do que veio antes.

2 Parte 2 — o conversor, em bruto

Tarefa 5. Com o driver fornecido, mostre no display o valor bruto do conversor, de 0 a 1023, atualizado a cada 500 ms. Confira que o driver habilita **só** o AN0 (ADCON1 = 0x0E) — a Tarefa 4 explica por quê.

TAREFA · 1,0 pt

	Previsto (P2)	Observado
Código à temperatura ambiente		
Código com o dedo no sensor, 10 s		
Variação por grau (P4)		

Da referência de 5 V e dos dez bits: $1 \text{ LSB} = 5000/1024 \approx 4,88 \text{ mV}$. E o LM35 entrega 10 mV por grau, então $1 \text{ LSB} \approx 0,488 \text{ }^\circ\text{C}$.

CONCEITO

Meio grau. Este é o degrau da sua bancada, e ele não melhora com código melhor. O manual informa que o LM35 fica montado junto à resistência de aquecimento. Ele mede a temperatura do conjunto sensor-resistência, e não a do ar da sala — uma diferença de alguns graus para mais é esperada, e não é erro de conversão. Comparar com um termômetro da sala e concluir que “a conta está errada” é o erro que este parágrafo existe para evitar.

NOTA

3 Parte 3 — a conversão, sem ponto flutuante

O LM35 entrega 10 mV por grau, e 1 mV equivale a 0,1 °C. Logo o valor em milivolts já é o valor em décimos de grau, e não é preciso converter duas vezes:

CONCEITO

$$\text{décimos de grau} = \text{código} \cdot 625/128$$

A fração é exata, e 128 é potência de dois — o que transforma a divisão num deslocamento.

Tarefa 6. Implemente e mostre a temperatura no formato NN.N °C, com largura fixa — como o R4 estabeleceu.

TAREFA · 2,0 pt

```
int16_t adc_para_decimos(uint16_t leitura)
{
    uint32_t acumulador = (uint32_t) leitura * 625UL;
    return (int16_t)(acumulador >> 7);
}
```

(a) Por que o acumulador precisa ser de 32 bits? Calcule o maior produto possível e compare com 65535.

(b) A partir de que temperatura o defeito apareceria, se o acumulador fosse de 16 bits?

(c) Troque o uint32_t por uint16_t de propósito, grave, e aqueça o sensor até passar do ponto que você calculou em (b). Registre o que a tela mostrou, e compare com a sua previsão.

Repare no que o programa **não** fez na alínea (c): não travou, não reiniciou e não avisou. Ele continuou exibindo um número com toda a confiança. É o modo de falha mais comum de aritmética inteira em sistema embarcado, e vocês vão reconhecê-lo pelo resto da vida.

ATENÇÃO

4 Parte 4 — o ruído, e o que a média não conserta

Tarefa 7. Com o kit imóvel e sem tocar no sensor, observe o último dígito por trinta segundos.

TAREFA · 1,5 pt

Previsão P3 dizia quantos códigos distintos? _____

Quantos você contou? _____

A oscilação observada é compatível com 1 LSB?

Agora implemente a média de oito amostras e observe de novo.

```
uint16_t adc_media(uint8_t n_pot2)
{
    uint32_t soma = 0;
    uint16_t total = (uint16_t)(1u << n_pot2);
    for (uint16_t i = 0; i < total; i++) {
        soma += adc_amostra();
    }
    return (uint16_t)(soma >> n_pot2);
}
```

A oscilação diminuiu? Desapareceu? E por que o número de amostras é potência de dois?

A média reduz o ruído aleatório, que é o que faz o dígito oscilar. Ela não reduz o erro de quantização, que é sistemático: se a tensão real cai entre dois códigos, nenhuma quantidade de médias inventa o valor intermediário.

CONCEITO

E há um caso que costuma surpreender: se a leitura for perfeitamente estável, a média não melhora nada — oito amostras idênticas têm média igual a elas mesmas. Um conversor limpo demais não pode ser melhorado por média.

Distinguir essas duas fontes de erro é o que separa quem filtra por hábito de quem filtra por motivo.

5 Entrega

As tabelas das Tarefas 2 e 5; o registro do sintoma provocado na Tarefa 3; o registro do estouro provocado na Tarefa 6; a contagem de códigos distintos da Tarefa 7.

TAREFA

Responda também:

(a) A partir do valor de ADCON1 que você leu, explique por que o botão respondia sem nenhuma configuração, e por que parou de responder com `ADCON1 = 0x00`. Descreva a causa em termos do que existe atrás do pino, e não em termos de “faltava uma linha”.

(b) A tensão de alimentação medida no início alimenta o PIC e o LM35, e serve de referência para o conversor. Se na sua bancada ela mediu 4,80 V em vez de 5,00 V, o degrau real do conversor é maior ou menor que 4,88 mV? O que acontece com a temperatura exibida?

(c) Você mediu quantos códigos distintos com o sistema parado. A média de oito melhorou? Se não melhorou, o que isso diz sobre o ruído da sua bancada?

6 Se sobrar tempo

E1 — a borda, de perto. Com o osciloscópio no pino do botão INT0, aperte e solte devagar, observando a transição. A borda é limpa? Descreva o que vê entre os dois níveis.

OPCIONAL

E2 — o custo da média. Com um pino auxiliar levantado antes e baixado depois, compare o tempo de uma leitura simples e o de uma média de oito.

OPCIONAL

	Tempo	Ciclos
Uma conversão		
Média de oito		
Atualização de tela (do R4)		

Qual das três domina o laço? Vale a pena fazer média de oito?

E3 — o custo da representação. Compile duas versões do mesmo programa: uma com a conversão inteira da Tarefa 6 e outra com

OPCIONAL

```
float temp = leitura * 5000.0f / 1024.0f / 10.0f;
```

Compare o tamanho do programa no arquivo de mapa gerado pela compilação.

	Inteira	Com float
Bytes de programa		
Percentual dos 32 kB		

O número que sai daqui é o que justifica, retroativamente, a decisão do R0 de representar temperatura como `int16_t` em décimos de grau. Não foi preferência estética nem purismo: é o preço medido, no seu compilador, do tipo que parecia mais natural.

No R6: o PWM, e o mesmo sinal com dois significados. A ventoinha vai ler a razão cíclica como energia, e o buzzer vai ler a frequência como nota — com o mesmo código, mudando só uma chave. E uma descoberta desconfortável: o módulo de PWM do chip não alcança o lá de 440 Hz que vocês tocaram no R3.

NOTA